Społeczna Akademia Nauk w Łodzi

Kierunek studiów: informatyka, studia inżynierskie I stopnia

semestr IV, studia niestacjonarne

Projekt z przedmiotu

Bazy danych

Praca wykonana pod kierunkiem: dr. inż. Józef Paszkowski

Tytuł projektu: Baza Danych Systemu Pogotowia Ratunkowego

Projekt został wykonany przez:

Damian Baliński, nr indeksu 123702 (udział 100%)

Spis treści

1. Cel projektu ... 2
2. Zakres projektu ... 3
3. Użytkownicy systemu ... 3
4. Wymagania funkcjonalne systemu ... 4
5. Ewolucja systemu ... 5
6. Wybrane komponenty oprogramowania ... 6
7. Opis systemu ... 7
   7.1 Diagram przypadków użycia ... 7
   7.2 Diagram encji (ERD) ... 7
   7.3 Struktura bazy danych - definicje tabel, klucze główne i obce ... 9
   7.4 Zapytania do bazy danych realizujące funkcje użytkowników ... 14
   7.5 Wyzwalacze (triggery) w systemie ... 20
   7.6 Testowanie bazy danych - przykładowe dane i wyniki zapytań ... 23
   7.7 Opis kont użytkowników systemu i uprawnień ... 29
8. Opis implementacji i wdrożenia projektu ... 30
   8.1 Instalacja i konfiguracja środowiska ... 30
   8.2 Tworzenie struktury bazy danych ... 31
   8.3 Implementacja zapytań, widoków i wyzwalaczy ... 31
   8.4 Implementacja systemu ról i uprawnień ... 31
9. Uwagi końcowe i wnioski ... 32
10. Bibliografia / Źródła ... 33

1. Cel projektu

Celem projektu jest zaprojektowanie i implementacja relacyjnej bazy danych obsługującej system informatyczny pogotowia ratunkowego. Baza danych stanowi fundament systemu zarządzania interwencjami medycznymi - umożliwia rejestrowanie zgłoszeń, koordynowanie wyjazdów karetek, dokumentowanie udzielonej pomocy medycznej oraz zarządzanie zasobami ludzkimi i sprzętowymi stacji pogotowia.

Projekt realizuje pełen cykl wytwórczy bazy danych: od analizy wymagań i modelowania konceptualnego, przez implementację struktury tabel i danych testowych, aż po zaawansowane mechanizmy takie jak widoki, wyzwalacze oraz system ról i uprawnień. Efektem końcowym jest w pełni funkcjonalna baza danych PostgreSQL gotowa do integracji z aplikacją frontendową.

Dodatkowym celem dydaktycznym projektu jest praktyczne zastosowanie zasad normalizacji do postaci trzeciej normalnej (3NF), projektowania diagramów ERD, pisania złożonych zapytań SQL z wielotabelowymi złączeniami, a także implementacji mechanizmów automatyzacji i kontroli dostępu charakterystycznych dla systemów produkcyjnych.

2. Zakres projektu

Projekt obejmuje następujące obszary funkcjonalne systemu pogotowia ratunkowego:

- **Zarządzanie zgłoszeniami** - rejestracja wezwań przyjmowanych przez dyspozytorów, wraz z lokalizacją, priorytetem i opisem zdarzenia
- **Zarządzanie wyjazdami** - przypisywanie karetek i zespołów ratowniczych do wezwań, śledzenie czasów wyjazdu, przybycia i powrotu
- **Dokumentacja medyczna** - ewidencja pacjentów, rejestracja procedur medycznych udzielonych w trakcie interwencji
- **Zarządzanie zasobami** - ewidencja karetek (z podziałem na typy), pracowników, zespołów ratowniczych i ich składów
- **Zarządzanie zmianami** - rejestracja dyżurów dyspozytorów
- **Katalog świadczeń medycznych** - standaryzowana lista procedur medycznych stosowanych przez ratowników
- **Raportowanie i analityka** - zestaw zapytań SQL i widoków umożliwiających generowanie raportów operacyjnych i zarządczych
- **Automatyzacja** - wyzwalacze automatycznie aktualizujące status karetek i rejestrujące historię zmian
- **Bezpieczeństwo** - system ról i uprawnień odzwierciedlający strukturę organizacyjną pogotowia

Zakres projektu nie obejmuje implementacji warstwy aplikacyjnej (interfejsu użytkownika) ani integracji z zewnętrznymi systemami (np. systemami szpitalnymi czy CAD).

3. Użytkownicy systemu

W systemie zdefiniowano trzy główne grupy użytkowników, różniące się zakresem dostępu i odpowiedzialności:

Dyspozytor - pracownik dyspozytorni przyjmujący zgłoszenia telefoniczne od osób wzywających pomoc. Dyspozytor rejestruje nowe wezwania w systemie, przypisuje im priorytet (P1/P2/P3) oraz śledzi status realizacji zgłoszeń. Ma dostęp do podglądu dostępności karetek i zespołów, jednak nie ingeruje w dane medyczne pacjentów.

Ratownik medyczny / Lekarz - członek zespołu ratowniczego obsługujący wyjazd. Korzysta z systemu (np. poprzez tablet w karetce) w celu rejestracji szczegółów wyjazdu, danych pacjenta oraz wykonanych procedur medycznych. Ma szerszy dostęp do danych operacyjnych niż dyspozytor, jednak nie może modyfikować struktury systemu ani zarządzać kontami.

Administrator systemu - osoba odpowiedzialna za utrzymanie technicznej sprawności bazy danych. Posiada pełny dostęp do wszystkich zasobów systemu, zarządza kontami użytkowników, wykonuje kopie zapasowe i monitoruje wydajność. W codziennej pracy operacyjnej rola administratora nie jest używana.

4. Wymagania funkcjonalne systemu

WF-01 - System musi umożliwiać rejestrację nowego wezwania wraz z adresem, godziną, priorytetem i opisem zdarzenia.

WF-02 - System musi umożliwiać przypisanie karetki i zespołu ratowniczego do wezwania oraz rejestrację czasów wyjazdu, przybycia na miejsce i powrotu.

WF-03 - System musi automatycznie aktualizować status karetki na „W trasie" przy rejestracji wyjazdu oraz na „Dostępna" po odnotowaniu powrotu.

WF-04 - System musi umożliwiać rejestrację danych osobowych pacjenta i powiązanie go z konkretnym wyjazdem.

WF-05 - System musi umożliwiać rejestrację procedur medycznych udzielonych pacjentowi w trakcie interwencji, na podstawie standaryzowanego katalogu świadczeń.

WF-06 - System musi przechowywać historię zmian statusu wezwań w celach audytowych.

WF-07 - System musi uniemożliwiać rejestrację wyjazdu z zespołem posiadającym mniej niż dwóch członków.

WF-08 - System musi umożliwiać generowanie raportów: historii wyjazdów karetki, procedur udzielonych pacjentowi, statystyk wezwań według priorytetu, obciążenia dyspozytorni oraz dostępności karetek w czasie rzeczywistym.

WF-09 - System musi kontrolować dostęp do danych zgodnie z rolą użytkownika, uniemożliwiając wykonywanie nieautoryzowanych operacji.

WF-10 - System musi obsługiwać katalog świadczeń medycznych z kodami procedur, nazwami i kategoriami.

5. Ewolucja systemu

Projekt realizowano iteracyjnie, przechodząc przez kolejne etapy:

Etap 0 - Konfiguracja środowiska: Instalacja PostgreSQL, konfiguracja klienta DBeaver, utworzenie bazy danych `pogotowie_db`, inicjalizacja repozytorium Git z podziałem na foldery `/sql/ddl/`, `/sql/dml/`, `/sql/queries/`, `/sql/views/`, `/sql/triggers/`, `/sql/roles/`.

Etap 1 - Analiza wymagań i model konceptualny: Identyfikacja encji domeny (dyspozytor, wezwanie, karetka, pracownik, zespół, wyjazd, pacjent, świadczenie medyczne) i ich wzajemnych relacji. Określenie typów relacji (1:N, N:M) i tabel łączących.

Etap 2 - Diagram ERD: Zaprojektowanie schematu 13 tabel w notacji DBML przy użyciu narzędzia dbdiagram.io. Weryfikacja zgodności ze strukturą 3NF.

Etap 3 - Implementacja DDL: Napisanie skryptów SQL tworzących tabele z constraintami CHECK, UNIQUE, NOT NULL, kluczami obcymi i indeksami na polach FK oraz polach wyszukiwania.

Etap 4 - Dane testowe DML: Wypełnienie wszystkich tabel realistycznymi danymi testowymi (5 dyspozytorów, 8 karetek, 15 pracowników, 6 zespołów, 20 wezwań, 15 wyjazdów, 10 pacjentów, 20 świadczeń medycznych).

Etap 5 - Zapytania SQL: Implementacja 8 zapytań realizujących wymagania funkcjonalne systemu.

Etap 6 - Widoki: Implementacja widoków upraszczających raportowanie i dostęp do danych.

Etap 7 - Wyzwalacze: Implementacja triggerów automatyzujących zarządzanie statusami i audyt zmian.

Etap 8 - Role i uprawnienia: Konfiguracja systemu kontroli dostępu z trzema rolami.

Etap 9 - Testowanie: Weryfikacja poprawności działania wszystkich komponentów systemu.

6. Wybrane komponenty oprogramowania

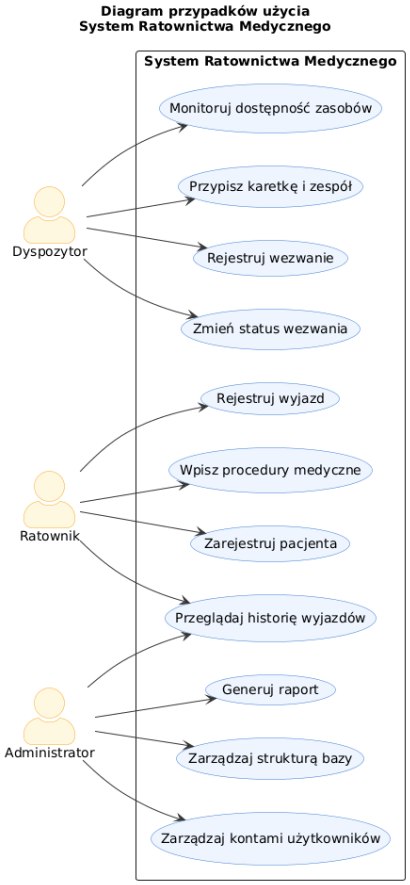
Komponent	Wersja	Rola w projekcie
PostgreSQL	16+	Silnik relacyjnej bazy danych
DBeaver Community	25.2.2	Klient SQL, zarządzanie bazą, wykonywanie skryptów
dbdiagram.io	online	Projektowanie diagramu ERD w notacji DBML
Visual Studio Code	1.112.0	Edytor skryptów SQL, zarządzanie plikami projektu
Git + GitHub	-	Kontrola wersji skryptów, historia zmian projektu

PostgreSQL został wybrany jako silnik bazy danych ze względu na pełne wsparcie standardu SQL, zaawansowane mechanizmy triggerów i procedur składowanych w języku PL/pgSQL, rozbudowany system ról i uprawnień, a także szerokie zastosowanie w sektorze ochrony zdrowia. Jest oprogramowaniem w pełni bezpłatnym i open-source, stanowiącym standard w branży.

DBeaver jako klient SQL umożliwia wygodne zarządzanie połączeniami, edycję i wykonywanie skryptów, przeglądanie struktury tabel oraz eksport wyników zapytań.

7. Opis systemu

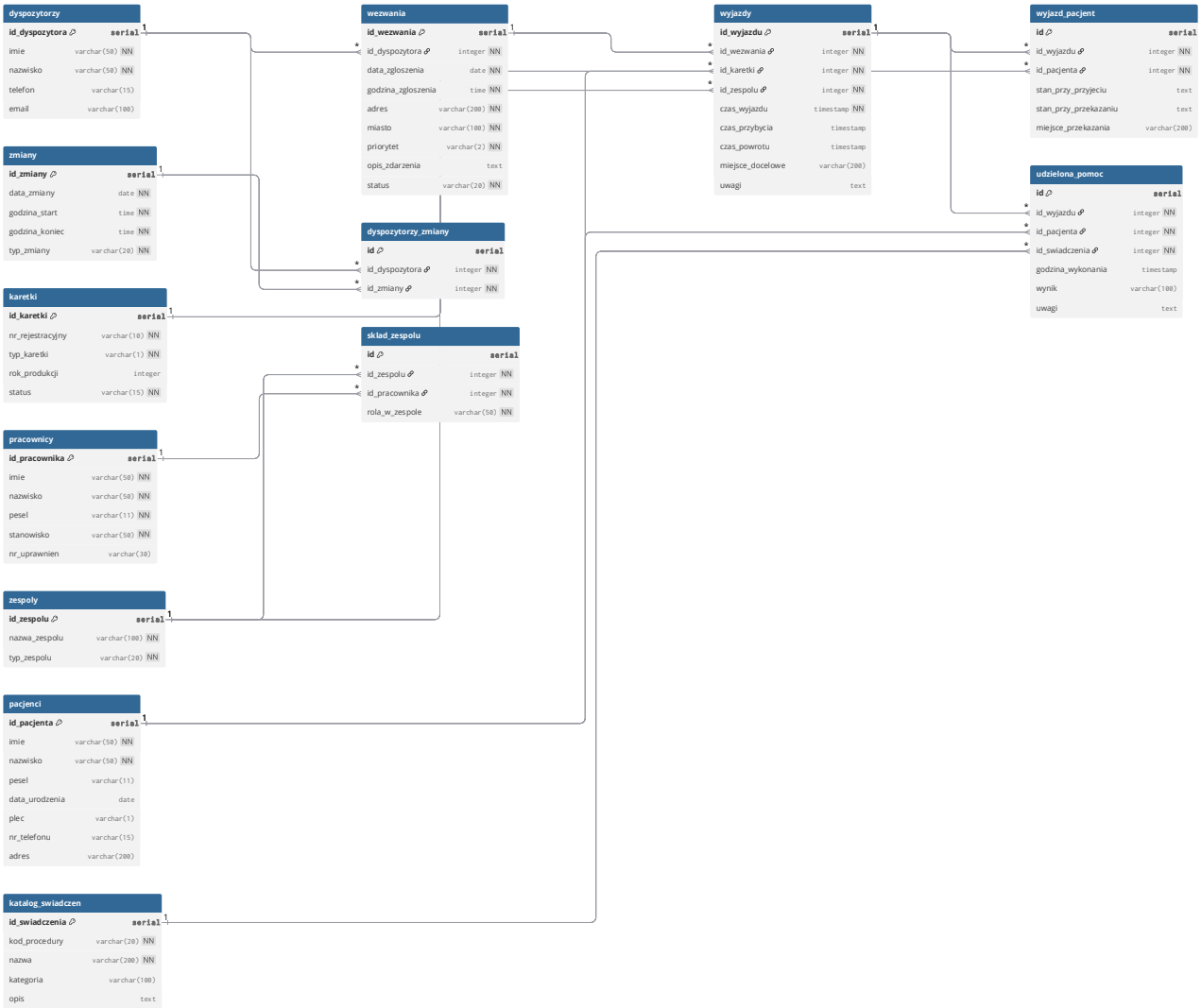
7.1 Diagram przypadków użycia



Dyspozytor wchodzi w interakcję z funkcjami rejestracji wezwań i monitorowania dostępności zasobów. Ratownik obsługuje rejestrację wyjazdów, pacjentów i świadczeń medycznych. Administrator zarządza całością systemu, kontami użytkowników i strukturą bazy.

7.2 Diagram encji (ERD)

Diagram ERD przedstawia 13 tabel systemu wraz z relacjami między nimi.



System oparty jest na 13 tabelach rozmieszczonych w trzech warstwach:

- **Warstwa zasobów:** dyspozytorzy, karetki, pracownicy, zespoly, zmiany
- **Warstwa operacyjna:** wezwania, wyjazdy, pacjenci
- **Warstwa łącząca i medyczna:** sklad_zespolu, dyspozytorzy_zmiany, wyjazd_pacjent, katalog_swiadczen, udzielona_pomoc

Wszystkie tabele spełniają wymagania **trzeciej postaci normalnej (3NF):** każdy atrybut zależy wyłącznie od klucza głównego tabeli. Eliminuje to redundancję danych - dane osobowe pracownika przechowywane są wyłącznie w tabeli pracownicy i nigdy nie powielane w sklad_zespolu. Katalog świadczeń jako osobna tabela katalog_swiadczen eliminuje powtarzanie nazw procedur przy każdym wpisie w udzielona_pomoc.

7.3 Struktura bazy danych - definicje tabel, klucze główne i obce

Poniżej przedstawiono definicje wszystkich 13 tabel systemu wraz z opisem ich roli.

Tabela: dyspozytorzy

Przechowuje dane pracowników dyspozytorni przyjmujących zgłoszenia.

```
CREATE TABLE dyspozytorzy (
    id_dyspozytora  SERIAL PRIMARY KEY,
    imie            VARCHAR(50)  NOT NULL,
    nazwisko        VARCHAR(50)  NOT NULL,
    telefon         VARCHAR(15)  UNIQUE,
    email           VARCHAR(100) UNIQUE
);
```

Tabela: karetki

Ewidencja pojazdów ratowniczych z podziałem na typy i statusy dostępności.

```
CREATE TABLE karetki (

    id_karetki        SERIAL PRIMARY KEY,
    nr_rejestracyjny  VARCHAR(10)  NOT NULL UNIQUE,
    typ_karetki       VARCHAR(1)   NOT NULL CHECK (typ_karetki IN ('P', 'S', 'R')),
    rok_produkcji     INTEGER      CHECK (rok_produkcji BETWEEN 1990 AND 2030),
    status            VARCHAR(15)  NOT NULL DEFAULT 'DOSTEPNA'
                                   CHECK (status IN ('DOSTEPNA', 'W_TRASIE', 'W_SERWISIE')) );
```

Typy karetek: **P** - podstawowy, **S** - specjalistyczny, **R** - reanimacyjny.

Tabela: pracownicy

Dane osobowe i kwalifikacje pracowników medycznych oraz kierowców.

```
CREATE TABLE pracownicy (

    id_pracownika  SERIAL PRIMARY KEY,
    imie           VARCHAR(50) NOT NULL,
    nazwisko       VARCHAR(50) NOT NULL,
    pesel          VARCHAR(11) NOT NULL UNIQUE,
    stanowisko     VARCHAR(50) NOT NULL
                   CHECK (stanowisko IN ('ratownik_medyczny', 'lekarz', 'kierowca')),
     nr_uprawnien   VARCHAR(30) UNIQUE

);
```

Tabela: `zespoly`

Nazwane jednostki organizacyjne - grupy ratownicze przypisywane do wyjazdów.

```
CREATE TABLE zespoly (
    id_zespolu    SERIAL PRIMARY KEY,
    nazwa_zespolu VARCHAR(100) NOT NULL,
    typ_zespolu   VARCHAR(20)  NOT NULL
                  CHECK (typ_zespolu IN ('podstawowy', 'specjalistyczny', 'reanimacyjny'))
);
```

Tabela: `sklad_zespolu`

Tabela łącząca N:M - przypisanie pracowników do zespołów wraz z rolą.

```
CREATE TABLE sklad_zespolu (
    id             SERIAL PRIMARY KEY,
    id_zespolu     INTEGER     NOT NULL REFERENCES zespoly(id_zespolu),
    id_pracownika  INTEGER     NOT NULL REFERENCES pracownicy(id_pracownika),
    rola_w_zespole VARCHAR(50) NOT NULL,
    UNIQUE (id_zespolu, id_pracownika)
);
CREATE INDEX idx_sklad_zespolu_zespol    ON sklad_zespolu(id_zespolu);
CREATE INDEX idx_sklad_zespolu_pracownik ON sklad_zespolu(id_pracownika);
```

Tabela: `zmiany`

Harmonogram dyżurów dyspozytorni (zmiany dzienne i nocne).

```
CREATE TABLE zmiany (
    id_zmiany      SERIAL PRIMARY KEY,
    data_zmiany    DATE        NOT NULL,
    godzina_start  TIME        NOT NULL,
    godzina_koniec TIME        NOT NULL,
    typ_zmiany     VARCHAR(20) NOT NULL CHECK (typ_zmiany IN ('dzienna', 'nocna'))
);
```

Tabela: `dyspozytorzy_zmiany`

Tabela łącząca N:M - przypisanie dyspozytorów do zmian.

```
CREATE TABLE dyspozytorzy_zmiany (
```

```
    id              SERIAL  PRIMARY KEY,
    id_dyspozytora  INTEGER NOT NULL REFERENCES dyspozytorzy(id_dyspozytora),
    id_zmiany       INTEGER NOT NULL REFERENCES zmiany(id_zmiany),
    UNIQUE (id_dyspozytora, id_zmiany)

);

CREATE INDEX idx_dyz_dyspozytor ON dyspozytorzy_zmiany(id_dyspozytora);

CREATE INDEX idx_dyz_zmiana     ON dyspozytorzy_zmiany(id_zmiany);
```

Tabela: wezwania

Centralna tabela zgłoszeń - rejestruje każde wezwanie przyjęte przez dyspozytora.

```
CREATE TABLE wezwania (

    id_wezwania        SERIAL PRIMARY KEY,
    id_dyspozytora     INTEGER      NOT NULL REFERENCES dyspozytorzy(id_dyspozytora),
    data_zgloszenia    DATE         NOT NULL,
    godzina_zgloszenia TIME         NOT NULL,
    adres              VARCHAR(200) NOT NULL,
    miasto             VARCHAR(100) NOT NULL,
    priorytet          VARCHAR(2)   NOT NULL CHECK (priorytet IN ('P1', 'P2', 'P3')),
    opis_zdarzenia     TEXT,
    status             VARCHAR(20)  NOT NULL DEFAULT 'NOWE'
                                    CHECK (status IN ('NOWE', 'W_REALIZACJI', 'ZAKONCZONE',
'ANULOWANE'))

);


CREATE INDEX idx_wezwania_dyspozytor ON wezwania(id_dyspozytora);

CREATE INDEX idx_wezwania_status     ON wezwania(status);

CREATE INDEX idx_wezwania_data       ON wezwania(data_zgloszenia);
```

Tabela: wyjazdy

Rejestruje realizację wezwania - przypisuje karetkę, zespół i dokumentuje czasy.

```
CREATE TABLE wyjazdy (


    id_wyjazdu       SERIAL PRIMARY KEY,
    id_wezwania      INTEGER      NOT NULL REFERENCES wezwania(id_wezwania),
    id_karetki       INTEGER      NOT NULL REFERENCES karetki(id_karetki),
    id_zespolu       INTEGER      NOT NULL REFERENCES zespoly(id_zespolu),
    czas_wyjazdu     TIMESTAMP    NOT NULL,
```

```
    czas_przybycia   TIMESTAMP,
    czas_powrotu     TIMESTAMP,
    miejsce_docelowe VARCHAR(200),
    uwagi            TEXT

);


CREATE INDEX idx_wyjazdy_wezwanie ON wyjazdy(id_wezwania);

CREATE INDEX idx_wyjazdy_karetka  ON wyjazdy(id_karetki);

CREATE INDEX idx_wyjazdy_zespol   ON wyjazdy(id_zespolu);
```

Tabela: pacjenci

Dane osobowe osób, którym udzielono pomocy medycznej.

```
CREATE TABLE pacjenci (


    id_pacjenta   SERIAL PRIMARY KEY,
    imie          VARCHAR(50)  NOT NULL,
    nazwisko      VARCHAR(50)  NOT NULL,
    pesel         VARCHAR(11)  UNIQUE,
    data_urodzenia DATE,
    plec          VARCHAR(1)   CHECK (plec IN ('M', 'K', 'N')),
    nr_telefonu   VARCHAR(15),
    adres         VARCHAR(200)

);
```

Tabela: `wyjazd_pacjent`

Tabela łącząca N:M - przypisanie pacjentów do wyjazdów z dokumentacją stanu zdrowia.

```
CREATE TABLE wyjazd_pacjent (

    id                    SERIAL PRIMARY KEY,
    id_wyjazdu            INTEGER NOT NULL REFERENCES wyjazdy(id_wyjazdu),
    id_pacjenta           INTEGER NOT NULL REFERENCES pacjenci(id_pacjenta),
    stan_przy_przyjeciu   TEXT,
    stan_przy_przekazaniu TEXT,
    miejsce_przekazania   VARCHAR(200),
    UNIQUE (id_wyjazdu, id_pacjenta)

);
```

```
CREATE INDEX idx_wp_wyjazd  ON wyjazd_pacjent(id_wyjazdu);

CREATE INDEX idx_wp_pacjent ON wyjazd_pacjent(id_pacjenta);
```

Tabela: `katalog_swiadczen`

Standaryzowany słownik procedur medycznych stosowanych przez zespoły ratownicze.

```
CREATE TABLE katalog_swiadczen (
    id_swiadczenia  SERIAL PRIMARY KEY,
    kod_procedury   VARCHAR(20)  NOT NULL UNIQUE,
    nazwa           VARCHAR(200) NOT NULL,
    kategoria       VARCHAR(100),
    opis            TEXT

);
```

Tabela: `udzielona_pomoc`

Rejestruje wykonanie konkretnej procedury medycznej u pacjenta podczas wyjazdu.

```
CREATE TABLE udzielona_pomoc (

    id              SERIAL PRIMARY KEY,
    id_wyjazdu      INTEGER   NOT NULL REFERENCES wyjazdy(id_wyjazdu),
    id_pacjenta     INTEGER   NOT NULL REFERENCES pacjenci(id_pacjenta),
    id_swiadczenia  INTEGER   NOT NULL REFERENCES katalog_swiadczen(id_swiadczenia),
    godzina_wykonania TIMESTAMP,
    wynik           VARCHAR(100),
    uwagi           TEXT

);


CREATE INDEX idx_up_wyjazd     ON udzielona_pomoc(id_wyjazdu);
CREATE INDEX idx_up_pacjent    ON udzielona_pomoc(id_pacjenta);
CREATE INDEX idx_up_swiadczenie ON udzielona_pomoc(id_swiadczenia);
```

Tabela: `log_wezwan`

Tabela audytowa - automatycznie wypełniana przez trigger przy każdej zmianie statusu wezwania.

```
CREATE TABLE log_wezwan (
    id           SERIAL PRIMARY KEY,
    id_wezwania  INTEGER      NOT NULL,
    stary_status VARCHAR(20),
    nowy_status  VARCHAR(20),
    zmieniono_o  TIMESTAMP    DEFAULT NOW(),
```

```
    uzytkownik   VARCHAR(100) DEFAULT CURRENT_USER
);
```

7.4 Zapytania do bazy danych realizujące funkcje użytkowników

Zaimplementowano 8 zapytań SQL odpowiadających realnym potrzebom operacyjnym systemu pogotowia. Każde zapytanie zapisano jako osobny plik w katalogu `/sql/queries/`.

Q1 - Historia wyjazdów karetki w danym miesiącu

Zapytanie umożliwia prześledzenie wszystkich wyjazdów konkretnej karetki w zadanym miesiącu. Stosowane do raportów eksploatacyjnych pojazdów i planowania przeglądów.

```
SELECT
    k.nr_rejestracyjny,
    k.typ_karetki,
    w.id_wyjazdu,
    wez.data_zgloszenia,
    wez.priorytet,
    wez.adres,
    w.czas_wyjazdu,
    w.czas_powrotu,
    w.miejsce_docelowe
FROM wyjazdy w
JOIN karetki  k   ON w.id_karetki  = k.id_karetki
JOIN wezwania wez ON w.id_wezwania = wez.id_wezwania
WHERE k.nr_rejestracyjny = 'LDZ 1C005'
  AND EXTRACT(MONTH FROM w.czas_wyjazdu) = 1
  AND EXTRACT(YEAR  FROM w.czas_wyjazdu) = 2025
ORDER BY w.czas_wyjazdu;
```

Techniki SQL: JOIN, WHERE, EXTRACT, DATE

Q2 - Wszystkie procedury udzielone pacjentowi (po PESEL)

Zapytanie generuje pełną historię interwencji medycznych dla wskazanego pacjenta. Stosowane przez lekarzy SOR przy przyjęciu pacjenta przekazanego przez pogotowie.

```
SELECT
    CONCAT(p.imie, ' ', p.nazwisko) AS pacjent,
    p.pesel,
    wez.data_zgloszenia,
    wez.adres                       AS miejsce_zdarzenia,
    ks.kod_procedury,
```

```
    ks.nazwa                        AS procedura,
    ks.kategoria,
    up.godzina_wykonania,
    up.wynik,
    up.uwagi
FROM udzielona_pomoc up
JOIN pacjenci p
    ON up.id_pacjenta = p.id_pacjenta
JOIN katalog_swiadczen ks
    ON up.id_swiadczenia = ks.id_swiadczenia
JOIN wyjazdy wyj
    ON up.id_wyjazdu = wyj.id_wyjazdu
JOIN wezwania weź
    ON wyj.id_wezwania = wez.id_wezwania
WHERE p.pesel = '59010198765'
ORDER BY up.godzina_wykonania;
```

Techniki SQL: JOIN wielotabelowy (5 tabel), WHERE na indeksowanym polu PESEL

Q3 - Średni czas interwencji według typu karetki

Zapytanie oblicza statystyki czasu trwania interwencji z podziałem na typy karetek. Stosowane do oceny wydajności operacyjnej i porównania typów pojazdów.

```
SELECT
    k.typ_karetki,
    COUNT(w.id_wyjazdu)   AS liczba_wyjazdow,
    ROUND(AVG(EXTRACT(EPOCH FROM (w.czas_powrotu - w.czas_wyjazdu)) / 60))
                          AS sredni_czas_min,
    ROUND(MIN(EXTRACT(EPOCH FROM (w.czas_powrotu - w.czas_wyjazdu)) / 60))
                          AS min_czas_min,
    ROUND(MAX(EXTRACT(EPOCH FROM (w.czas_powrotu - w.czas_wyjazdu)) / 60))
                          AS max_czas_min
FROM wyjazdy w
JOIN karetki k ON w.id_karetki = k.id_karetki
WHERE w.czas_powrotu IS NOT NULL
GROUP BY k.typ_karetki
ORDER BY sredni_czas_min DESC;
```

Techniki SQL: AVG, MIN, MAX, EXTRACT(EPOCH FROM ...), GROUP BY

Q4 - Liczba wezwań według priorytetu i miesiąca

Zapytanie generuje miesięczne zestawienie wezwań z podziałem na priorytety. Podstawa raportów zarządczych i planowania zasobów.

```
WITH wezwania_miesiac AS (
    SELECT
        TO_CHAR(data_zgloszenia, 'YYYY-MM') AS miesiac,
        Priorytet
    FROM wezwania
)
SELECT
    miesiac,
    priorytet,
    COUNT(*) AS liczba_wezwan
FROM wezwania_miesiac
GROUP BY miesiac, priorytet
ORDER BY miesiac, priorytet;
```

Techniki SQL: GROUP BY, COUNT, TO_CHAR, CTE

Q5 - Dyspozytorzy z liczbą przyjętych zgłoszeń

Zapytanie zestawia obciążenie każdego dyspozytora z podziałem na priorytety zgłoszeń. Stosowane do oceny pracy dyspozytorni i planowania zmian.

```
SELECT
    CONCAT(d.imie, ' ', d.nazwisko)     AS dyspozytor,
    d.email,
    COUNT(wez.id_wezwania)          AS liczba_zgloszen,
    COUNT(CASE WHEN wez.priorytet = 'P1' THEN 1 END) AS zgloszenia_P1,
    COUNT(CASE WHEN wez.priorytet = 'P2' THEN 1 END) AS zgloszenia_P2,
    COUNT(CASE WHEN wez.priorytet = 'P3' THEN 1 END) AS zgloszenia_P3
FROM dyspozytorzy d
LEFT JOIN wezwania wez
    ON d.id_dyspozytora = wez.id_dyspozytora
GROUP BY d.id_dyspozytora, d.imie, d.nazwisko, d.email
ORDER BY liczba_zgloszen DESC;
```

Techniki SQL: LEFT JOIN, GROUP BY, COUNT, CASE

Q6 - Wyjazdy bez przypisanego pacjenta

Zapytanie identyfikuje wyjazdy, do których nie zarejestrowano żadnego pacjenta. Stosowane do kontroli kompletności danych - może wskazywać na przypadki odmowy transportu lub braki w dokumentacji.

```
SELECT
    w.id_wyjazdu,
    wez.data_zgloszenia,
    wez.adres,
    wez.priorytet,
    wez.opis_zdarzenia,
    k.nr_rejestracyjny,
    w.czas_wyjazdu,
    w.uwagi
FROM wyjazdy w
JOIN wezwania wez         ON w.id_wezwania = wez.id_wezwania
JOIN karetki k            ON w.id_karetki  = k.id_karetki
LEFT JOIN wyjazd_pacjent wp ON w.id_wyjazdu = wp.id_wyjazdu
WHERE wp.id_wyjazdu IS NULL
ORDER BY w.czas_wyjazdu;
```

Techniki SQL: LEFT JOIN, IS NULL (wzorzec antyjoin)

Q7 - Najczęściej stosowane procedury medyczne

Zapytanie rankinguje procedury według liczby wykonań, z udziałem procentowym. Stosowane do planowania zakupów sprzętu i leków.

```
SELECT
    ks.kod_procedury,
    ks.nazwa                    AS procedura,
    ks.kategoria,
    COUNT(up.id)                AS liczba_wykonan,
    ROUND(COUNT(up.id) * 100.0 /
          SUM(COUNT(up.id)) OVER (), 1) AS procent_udzialu
FROM katalog_swiadczen ks
LEFT JOIN udzielona_pomoc up ON ks.id_swiadczenia = up.id_swiadczenia
GROUP BY ks.id_swiadczenia, ks.kod_procedury, ks.nazwa, ks.kategoria
ORDER BY liczba_wykonan DESC;
```

Techniki SQL: GROUP BY, ORDER BY DESC, COUNT, funkcja okna SUM() OVER()

Q8 - Aktualnie dostępne karetki

Zapytanie zwraca listę karetek gotowych do wyjazdu - ze statusem DOSTEPNA i bez aktywnego, niezakończonego wyjazdu.

```
SELECT
    k.id_karetki,
    k.nr_rejestracyjny,
    k.typ_karetki,
    k.rok_produkcji,
    k.status
FROM karetki k
WHERE k.status = 'DOSTEPNA'
  AND k.id_karetki NOT IN (
      SELECT w.id_karetki
      FROM wyjazdy w
      WHERE w.czas_powrotu IS NULL   -- wyjazd trwa, karetka nie wróciła
  )
ORDER BY k.id_karetki;
```

Techniki SQL: podzapytanie skorelowane, IS NULL

Widoki (Views)

Oprócz zapytań zaimplementowano trzy widoki trwale zapisane w bazie danych, upraszczające dostęp do najczęściej potrzebnych zestawień.

1. v_szczegoly_wyjazdu

```
CREATE OR REPLACE VIEW v_szczegoly_wyjazdu AS

SELECT
    w.id_wyjazdu,
    -- dane zgłoszenia
    wez.id_wezwania,
    wez.data_zgloszenia,
    wez.godzina_zgloszenia,
    wez.adres               AS adres_zdarzenia,
    wez.miasto,
    wez.priorytet,
    wez.opis_zdarzenia,
    wez.status              AS status_wezwania,
    -- dyspozytor który przyjął zgłoszenie
    CONCAT(d.imie, ' ', d.nazwisko)     AS dyspozytor,
    -- karetka
    k.nr_rejestracyjny,
```

```
    k.typ_karetki,
    -- zespół
    z.nazwa_zespolu,
    z.typ_zespolu           AS typ_zespolu,
    -- czasy wyjazdu
    w.czas_wyjazdu,
    w.czas_przybycia,
    w.czas_powrotu,
    -- obliczony czas interwencji w minutach
    CASE
    WHEN w.czas_powrotu IS NOT NULL
    THEN ROUND(EXTRACT(EPOCH FROM (w.czas_powrotu - w.czas_wyjazdu)) / 60)
    END AS czas_interwencji_min,
    w.miejsce_docelowe,
    w.uwagi
FROM wyjazdy w
JOIN wezwania     wez ON w.id_wezwania  = wez.id_wezwania
JOIN karetki      k   ON w.id_karetki   = k.id_karetki
JOIN zespoly      z   ON w.id_zespolu   = z.id_zespolu
JOIN dyspozytorzy d   ON wez.id_dyspozytora = d.id_dyspozytora;
```

Widok zwraca szczegółowe informacje o wyjazdach zespołów ratowniczych, łącząc dane zgłoszenia, dyspozytora, karetki i zespołu. Dodatkowo oblicza czas trwania interwencji w minutach na podstawie czasu wyjazdu i powrotu.

2. v_historia_pacjenta

```
CREATE OR REPLACE VIEW v_historia_pacjenta AS

SELECT
    -- dane pacjenta
    p.id_pacjenta,
    CONCAT(p.imie, ' ', p.nazwisko)     AS pacjent,
    p.pesel,
    p.data_urodzenia,
    p.plec,
    -- zdarzenie
    wez.data_zgloszenia,
    wez.adres                       AS miejsce_zdarzenia,
    wez.priorytet,
    -- szczegóły wyjazdu i przekazania
    wp.stan_przy_przyjeciu,
    wp.stan_przy_przekazaniu,
```

```
    wp.miejsce_przekazania,
    -- procedura medyczna
    ks.kod_procedury,
    ks.nazwa                        AS procedura,
    ks.kategoria                    AS kategoria_procedury,
    up.godzina_wykonania,
    up.wynik,
    up.uwagi                        AS uwagi_procedury
FROM pacjenci p
JOIN udzielona_pomoc  up  ON p.id_pacjenta      = up.id_pacjenta
JOIN katalog_swiadczen ks  ON up.id_swiadczenia  = ks.id_swiadczenia
JOIN wyjazdy          wyj  ON up.id_wyjazdu      = wyj.id_wyjazdu
JOIN wezwania         wez  ON wyj.id_wezwania    = wez.id_wezwania
LEFT JOIN wyjazd_pacjent wp ON (wp.id_wyjazdu    = wyj.id_wyjazdu
                            AND wp.id_pacjenta   = p.id_pacjenta);
```

Widok zwraca historię medyczną pacjenta, obejmującą jego dane osobowe, informacje o zdarzeniach oraz wykonane procedury medyczne wraz z ich wynikami i szczegółami przekazania pacjenta.

3. v_obciazenie_dyspozytorni

```
CREATE OR REPLACE VIEW v_obciazenie_dyspozytorni AS

SELECT
    CONCAT(d.imie, ' ', d.nazwisko)     AS dyspozytor,
    TO_CHAR(wez.data_zgloszenia, 'YYYY-MM') AS miesiac,
    COUNT(wez.id_wezwania)          AS liczba_zgloszen,
    COUNT(CASE WHEN wez.priorytet = 'P1' THEN 1 END) AS p1,
    COUNT(CASE WHEN wez.priorytet = 'P2' THEN 1 END) AS p2,
    COUNT(CASE WHEN wez.priorytet = 'P3' THEN 1 END) AS p3,
    COUNT(CASE WHEN wez.status = 'ZAKONCZONE' THEN 1 END) AS zakończone
FROM dyspozytorzy d
LEFT JOIN wezwania wez ON d.id_dyspozytora = wez.id_dyspozytora
GROUP BY d.id_dyspozytora, d.imie, d.nazwisko,
         TO_CHAR(wez.data_zgloszenia, 'YYYY-MM');
```

Widok zwraca zestawienie obciążenia dyspozytorów w podziale na miesiące, prezentując liczbę obsłużonych zgłoszeń oraz ich strukturę według priorytetów i statusu zakończenia.

7.5 Wyzwalacze (triggery) w systemie

Zaimplementowano trzy wyzwalacze automatyzujące kluczowe operacje systemu.

Trigger T1 - Automatyczna zmiana statusu karetki

Eliminacja konieczności ręcznej aktualizacji statusu karetki przez użytkownika. System automatycznie zarządza dostępnością pojazdów.

```
-- Funkcja triggera


CREATE OR REPLACE FUNCTION fn_aktualizuj_status_karetki()
RETURNS TRIGGER AS $$
BEGIN



    -- NOWY WYJAZD -> karetka w trasie

    IF TG_OP = 'INSERT' THEN
        UPDATE karetki
        SET status = 'W_TRASIE'
        WHERE id_karetki = NEW.id_karetki;

    -- POWRÓT KARETKI
    ELSIF TG_OP = 'UPDATE'
       AND NEW.czas_powrotu IS NOT NULL
       AND OLD.czas_powrotu IS NULL THEN

        UPDATE karetki
        SET status = 'DOSTEPNA'
        WHERE id_karetki = NEW.id_karetki;

    END IF;

    RETURN NEW;


END;
$$ LANGUAGE plpgsql;



-- Definicja triggera

CREATE TRIGGER trg_status_karetki
AFTER INSERT OR UPDATE ON wyjazdy
FOR EACH ROW
EXECUTE FUNCTION fn_aktualizuj_status_karetki();
```

Mechanizm działania: Zmienna `TG_OP` rozróżnia typ operacji. Przy INSERT status karetki zmienia

się na W_TRASIE. Przy UPDATE sprawdzany jest warunek: jeśli pole czas_powrotu właśnie zostało uzupełnione (było NULL, teraz ma wartość), karetka wraca do statusu DOSTEPNA. Dzięki temu zapytanie Q8 zawsze zwraca aktualną listę dostępnych pojazdów.

Trigger T2 - Log zmian statusu wezwania

Rejestracja historii każdej zmiany statusu wezwania dla celów audytowych - wymóg w systemach obsługi danych medycznych.

```
-- funkcja
CREATE OR REPLACE FUNCTION fn_loguj_zmiane_statusu()
RETURNS TRIGGER AS $$
BEGIN
    -- Reaguj TYLKO gdy status faktycznie się zmienił
    IF NEW.status <> OLD.status THEN
        INSERT INTO log_wezwan (id_wezwania, stary_status, nowy_status,
                                zmieniono_o, uzytkownik)
        VALUES (OLD.id_wezwania, OLD.status, NEW.status,
                NOW(), CURRENT_USER);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- trigger
CREATE OR REPLACE TRIGGER trg_log_statusu_wezwania
AFTER UPDATE ON wezwania
FOR EACH ROW
EXECUTE FUNCTION fn_loguj_zmiane_statusu();
```

Mechanizm działania: Trigger reaguje wyłącznie gdy wartość kolumny status faktycznie się zmienia (NEW.status <> OLD.status). Dzięki temu aktualizacje innych pól wezwania (np. korekta adresu) nie generują wpisów w logu. Każdy wpis zawiera timestamp zmiany i identyfikator konta PostgreSQL, które wykonało operację.

Trigger T3 - Walidacja minimalnego składu zespołu

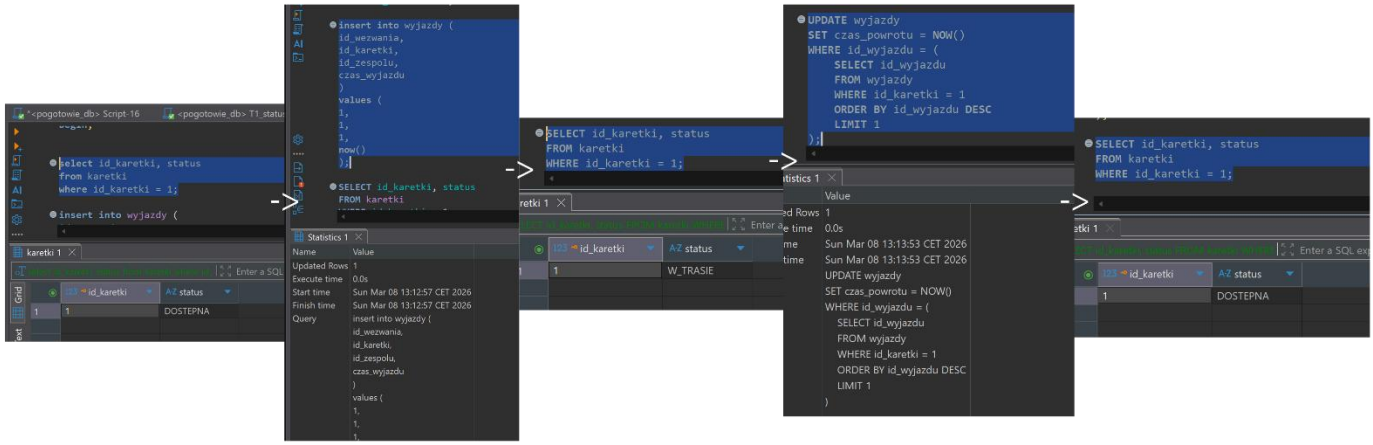
Zabezpieczenie przed wysłaniem wyjazdu z niewystarczającym składem zespołu ratowniczego. Wymóg bezpieczeństwa medycznego - każdy wyjazd musi obsługiwać minimum 2 osoby.

```
-- funkcja

CREATE OR REPLACE FUNCTION fn_waliduj_sklad_zespolu()
RETURNS TRIGGER AS $$
DECLARE
    v_liczba_czlonkow INTEGER;  -- zmienna przechowująca liczbę członków zespołu
BEGIN
    -- policz członków zespołu przypisanego do nowego wyjazdu
    SELECT COUNT(*) INTO v_liczba_czlonkow

    FROM sklad_zespolu
    WHERE id_zespolu = NEW.id_zespolu;

    -- jeśli mniej niż 2, przerwij INSERT i zgłoś błąd
    IF v_liczba_czlonkow < 2 THEN
        RAISE EXCEPTION
            'Zespół % ma tylko % członków. Wymagane minimum to 2.',
            NEW.id_zespolu, v_liczba_czlonkow;
    END IF;

    RETURN NEW;  -- pozwala na wykonanie INSERT jeśli warunek spełniony
END;
$$ LANGUAGE plpgsql;

-- trigger
CREATE OR REPLACE TRIGGER trg_walidacja_zespolu
BEFORE INSERT ON wyjazdy  -- trigger BEFORE, bo chcemy zatrzymać wstawienie, jeśli warunek nie
spełniony
FOR EACH RO
EXECUTE FUNCTION fn_waliduj_sklad_zespolu();
```

Mechanizm działania: Trigger typu BEFORE wykonuje się przed fizycznym zapisem wiersza do tabeli. Jeśli warunek składu nie jest spełniony, instrukcja `RAISE EXCEPTION` przerywa całą operację i zwraca komunikat błędu do aplikacji wywołującej. Wiersz nie trafia do bazy.

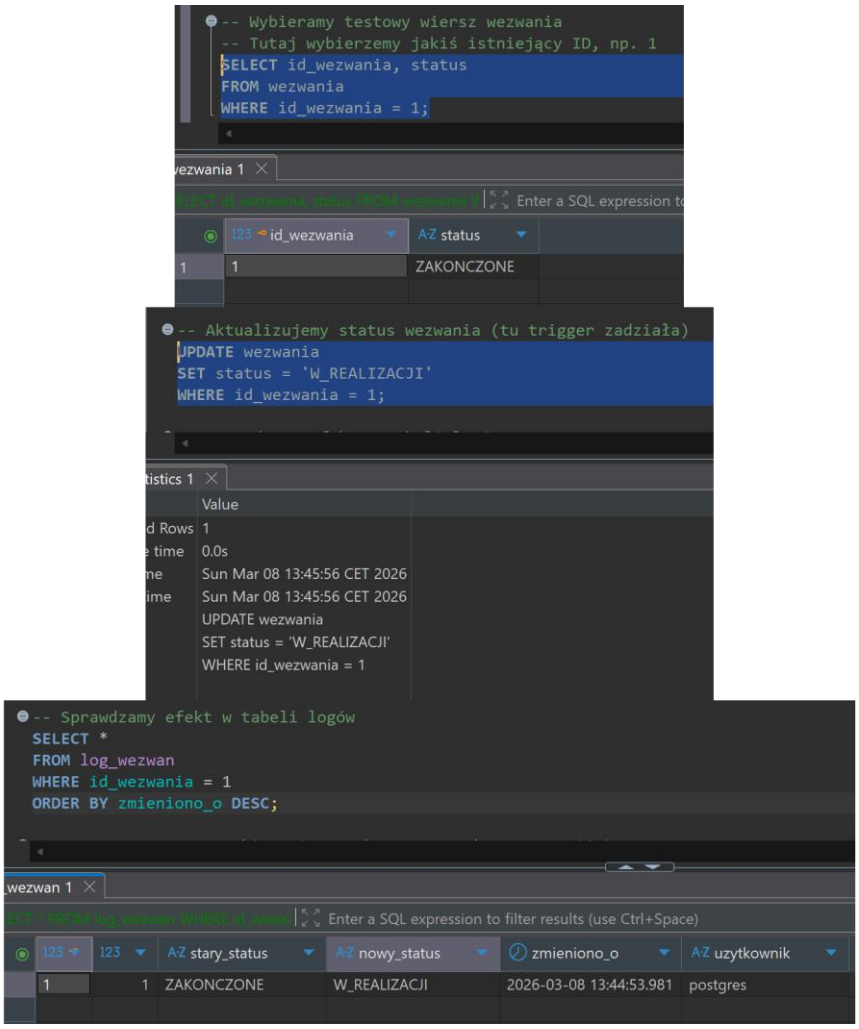
7.6 Testowanie bazy danych - przykładowe dane i wyniki zapytań

Test Triggera 1



Test weryfikuje działanie triggera trg_status_karetki wykonywanego po operacjach INSERT i UPDATE na tabeli wyjazdy. W kroku pierwszym potwierdzono wyjściowy stan karetki o id = 1 - status DOSTEPNA. Następnie zarejestrowano nowy wyjazd (INSERT), co automatycznie zmieniło status karetki na W_TRASIE bez żadnej ręcznej aktualizacji tabeli karetki. W kroku trzecim uzupełniono pole czas_powrotu (UPDATE), co trigger zinterpretował jako powrót karetki i przywrócił status DOSTEPNA. Test potwierdza poprawne działanie obu gałęzi logiki w funkcji fn_aktualizuj_status_karetki() - reagującej na TG_OP = 'INSERT' oraz na UPDATE z przejściem czas_powrotu z NULL na wartość.

Test Trigger 2



Test weryfikuje działanie triggera trg_log_statusu_wezwania, który automatycznie rejestruje każdą zmianę statusu wezwania w tabeli audytowej log_wezwan. W kroku pierwszym odczytano aktualny status wezwania o id = 1 - wartość ZAKONCZONE. Następnie wykonano UPDATE zmieniający status na W_REALIZACJI. Natychmiast po tej operacji w tabeli log_wezwan pojawił się nowy wpis zawierający poprzednią wartość pola (ZAKONCZONE), nową wartość (W_REALIZACJI), precyzyjny timestamp zmiany oraz identyfikator konta PostgreSQL, które wykonało operację (postgres). Test potwierdza poprawne działanie funkcji fn_loguj_zmiane_statusu() i spełnienie wymagania funkcjonalnego WF-06 dotyczącego audytu zmian.

Test Trigger 3

```
--  Utwórz pusty zespół testowy
INSERT INTO zespoly (nazwa_zespolu, typ_zespolu)
VALUES ('Pusty-Test', 'podstawowy');
```

stics 1

	Value
Rows	1
time	0.0s
e	Sun Mar 08 14:21:39 CET 2026
me	Sun Mar 08 14:21:39 CET 2026
	-- Utwórz pusty zespół testowy
	INSERT INTO zespoly (nazwa_zespolu, typ_zespolu)
	VALUES ('Pusty-Test', 'podstawowy')

```
--  Spróbuj wstawić wyjazd z pustym zespołem (powinno rzucić błąd triggera)
INSERT INTO wyjazdy (id_wezwania, id_karetki, id_zespolu, czas_wyjazdu, miejsce_docelowe)
VALUES (19, 2, 7, NOW(), 'Test');
--  Powinien zwrócić błąd: "Zespół 7 ma tylko 0 członków. Wymagane minimum to 2."
```

atistics 1

```
SQL Error [P0001]: BŁĄD: Zespół 7 ma tylko 0 członków.
Wymagane minimum to 2.
  Where: funkcja PL/pgSQL fn_waliduj_sklad_zespolu(),
wiersz 12 w RAISE
```

```
--  Spróbuj wstawić wyj
INSERT INTO wyjazdy (id
VALUES (19, 2, 7, NOW()
--  Powinien zwrócić bł
```

Test weryfikuje działanie triggera trg_walidacja_zespolu typu BEFORE INSERT, który blokuje rejestrację wyjazdu, gdy przypisany zespół liczy mniej niż dwóch członków. W celu przeprowadzenia testu utworzono pomocniczy pusty zespół 'Pusty-Test' o id = 7, niezawierający żadnych wpisów w tabeli sklad_zespolu. Następnie podjęto próbę zarejestrowania wyjazdu z przypisaniem do tego zespołu - trigger wykonał się przed fizycznym zapisem wiersza, policzył członków zespołu (wynik: 0) i wywołał RAISE EXCEPTION z komunikatem Zespół 7 ma tylko 0 członków.
Wymagane minimum to 2. Wiersz nie trafił do tabeli wyjazdy. Test potwierdza spełnienie wymagania funkcjonalnego WF-07 oraz skuteczność walidacji na poziomie bazy danych, niezależnie od warstwy aplikacyjnej

Test - Walidacja constraintu CHECK (pole priorytet)

```
INSERT INTO wezwania (id_dyspozytora, data_zgloszenia, godzina_zgloszenia,
                      adres, miasto, priorytet)
VALUES (1, CURRENT_DATE, CURRENT_TIME, 'ul. Testowa 1', 'Łódź', 'P4');
```

Celem testu jest weryfikacja, że baza danych odrzuca wartości niespełniające constraintu CHECK (priorytet IN ('P1', 'P2', 'P3')) zdefiniowanego na tabeli wezwania. Wykonano próbę wstawienia rekordu z wartością priorytet = 'P4', która nie należy do dopuszczalnego zbioru. Baza danych odrzuciła operację z błędem violates check constraint, nie dopuszczając do zapisu nieprawidłowej danej. Test potwierdza, że mechanizmy walidacji na poziomie schematu działają niezależnie od warstwy aplikacyjnej.

Wynik testu:

```
SQL Error [23514]: BŁĄD: nowy rekord dla relacji "wezwania" narusza ograniczenie
sprawdzające "wezwania_priorytet_check"
  Detail: Niepoprawne ograniczenia wiersza (21, 1, 2026-03-25, 11:33:13.832733,
ul. Testowa 1, Łódź, P4, null, NOWE).

Error position:
```

Test - Naruszenie klucza obcego (FK)

```
INSERT INTO wyjazdy (id_wezwania, id_karetki, id_zespolu, czas_wyjazdu)
VALUES (1, 999, 1, NOW());
```

Celem testu jest weryfikacja mechanizmu integralności referencyjnej (ang. *referential integrity*) pomiędzy tabelami wyjazdy i karetki. Wykonano próbę zarejestrowania wyjazdu z przypisaniem do nieistniejącej karetki (id_karetki = 999). PostgreSQL odrzucił operację, zgłaszając naruszenie constraintu klucza obcego wyjazdy_id_karetki_fkey. Test potwierdza, że w systemie niemożliwe jest pojawienie się „osieroconych" rekordów odwołujących się do nieistniejących zasobów.

Wynik testu:

```
Statistics 1
SQL Error [23503]: BŁĄD: wstawianie lub modyfikacja na tabeli "wyjazdy" narusza
klucz obcy "wyjazdy_id_karetki_fkey"
  Detail: Klucz (id_karetki)=(999) nie występuje w tabeli "karetki".
```

Test - Poprawność działania widoku v_szczegoly_wyjazdu

```
SELECT
    id_wyjazdu,
    data_zgloszenia,
    priorytet,
    adres_zdarzenia,
    nr_rejestracyjny
    typ_karetki,
    nazwa_zespolu,
    czas_wyjazdu,
    czas_powrotu,
    czas_interwencji_min
FROM v_szczegoly_wyjazdu
ORDER BY czas_wyjazdu DESC
LIMIT 5;
```

Celem testu jest weryfikacja, że widok v_szczegoly_wyjazdu poprawnie łączy dane z tabel wyjazdy, wezwania, karetki, zespoly i dyspozytorzy, a obliczana kolumna czas_interwencji_min zwraca prawidłowe wartości.
Wykonano zapytanie SELECT zwracające 5 ostatnich wyjazdów posortowanych malejąco po czasie wyjazdu. Wynik prezentuje w jednym miejscu dane operacyjne i logistyczne wyjazdu, potwierdzając poprawność JOIN-ów oraz kalkulacji czasu trwania interwencji. Widok upraszcza dostęp do danych i stanowi podstawę raportowania operacyjnego.

Wynik testu:

	123	data_zg	A-Z	A-Z adres_zdarzenia	A-Z nr_rejestr	A-Z typ	A-Z nazwa_ze	czas_wyjazdu	czas_powrotu	123 czas_inter
1	15	2025-01-15	P2	ul. Przybyszewskiego 3	LDZ 1A002	P	Alfa-2	2025-01-15 10:28:00.000	2025-01-15 11:30:00.000	62
2	14	2025-01-15	P1	ul. Tuszyńska 61	LDZ 1C006	R	Gamma-1	2025-01-15 08:08:00.000	2025-01-15 09:30:00.000	82
3	13	2025-01-14	P3	ul. Pomorska 88	LDZ 1A001	P	Alfa-1	2025-01-14 15:08:00.000	2025-01-14 16:00:00.000	52
4	12	2025-01-14	P2	ul. Aleksandrowska 3	LDZ 1B004	S	Beta-2	2025-01-14 11:38:00.000	2025-01-14 12:45:00.000	67
5	11	2025-01-13	P3	ul. Łagiewnicka 44	LDZ 1A001	P	Alfa-1	2025-01-13 18:08:00.000	2025-01-13 19:00:00.000	52

Test - Weryfikacja systemu uprawnień roli

```
-- Krok 1: przełącz na rolę dyspozytora
SET ROLE dyspozytor;

-- Krok 2: dozwolone — SELECT na wezwaniach
SELECT id_wezwania, priorytet, status, miasto
FROM wezwania
LIMIT 5;

-- Krok 3: niedozwolone — próba usunięcia danych
DELETE FROM wezwania WHERE id_wezwania = 1;
```

Celem testu jest potwierdzenie, że rola dyspozytor działa zgodnie z zasadą najmniejszych uprawnień - posiada dostęp do odczytu danych operacyjnych, lecz nie może modyfikować ani usuwać rekordów. Po przełączeniu kontekstu sesji na rolę dyspozytor (polecenie SET ROLE dyspozytor) wykonano dwie operacje: zapytanie SELECT na tabeli wezwania (zakończone sukcesem) oraz próbę DELETE na tym samym rekordzie. Operacja usunięcia została zablokowana przez PostgreSQL z komunikatem permission denied for table wezwania.

Wynik testu po SELECT:

	123 id_wezwania	A-Z priorytet	A-Z status	A-Z miasto
1	1	P1	ZAKONCZONE	Łódź
2	2	P2	ZAKONCZONE	Łódź
3	3	P1	ZAKONCZONE	Łódź
4	4	P3	ZAKONCZONE	Łódź
5	5	P2	ZAKONCZONE	Łódź

Wynik testu po DELETE:

```
SQL Error [42501]: BŁĄD: permission denied for table wezwania
```

7.7 Opis kont użytkowników systemu i uprawnień

W systemie zdefiniowano trzy role PostgreSQL odzwierciedlające strukturę organizacyjną pogotowia. Implementacja opiera się na zasadzie najmniejszych uprawnień (ang. *principle of least privilege*) - każda rola otrzymuje wyłącznie te uprawnienia, które są niezbędne do wykonywania jej obowiązków.

Rola: dyspozytor

Może odczytywać wszystkie dane operacyjne oraz tworzyć nowe wezwania i wyjazdy, a także aktualizować wyłącznie status wezwania oraz przypisanie karetki i zespołu. Nie ma możliwości modyfikacji danych medycznych, pacjentów ani pełnej edycji wyjazdów poza określonymi kolumnami.

Rola: ratownik

Może odczytywać dane w systemie oraz rejestrować i aktualizować wyjazdy, dane pacjentów, powiązania pacjent-wyjazd oraz udzieloną pomoc medyczną. Nie posiada uprawnień do usuwania danych ani zarządzania strukturą bazy i użytkownikami.

Rola: admin_pogotowie

Posiada pełne uprawnienia administracyjne do bazy danych, w tym możliwość zarządzania rolami, użytkownikami oraz tworzenia i modyfikowania baz danych. Ma uprawnienia SUPERUSER, co oznacza brak ograniczeń dostępu i kontroli uprawnień.

Skrypt definiujący role i nadający im uprawnienia został zapisany pod nazwą 01_role_uprawnienia.sql i znajduje się w repozytorium pod ścieżką pogotowie_db/sql/roles.

8. Opis implementacji i wdrożenia projektu

8.1 Instalacja i konfiguracja środowiska

Projekt realizowano w środowisku lokalnym na systemie operacyjnym Windows. Proces konfiguracji obejmował następujące kroki:

1. Pobranie i instalacja **PostgreSQL 16** ze strony postgresql.org - podczas instalacji skonfigurowano port 5432, hasło dla konta `postgres` oraz inicjalizację klastra bazy danych.
2. Instalacja **DBeaver Community** - po uruchomieniu skonfigurowano połączenie z lokalnym serwerem PostgreSQL (host: localhost, port: 5432, użytkownik: postgres).
3. Utworzenie bazy danych `pogotowie_db`
4. Inicjalizacja repozytorium Git z następującą strukturą katalogów:

```
pogotowie-db/
├── sql/
│   ├── ddl/        (skrypty tworzące strukturę tabel)
│   ├── dml/        (skrypty wstawiające dane testowe)
│   ├── queries/    (zapytania Q1-Q8)
│   ├── views/      (widoki)
│   ├── triggers/   (wyzwalacze)
│   └── roles/      (role i uprawnienia)
├── docs/           (sprawozdanie)
├── .gitignore
└── README.md
```

5. Opublikowanie repozytorium na platformie **GitHub**.

8.2 Tworzenie struktury bazy danych

Struktura bazy danych została zaimplementowana poprzez wykonanie 13 skryptów DDL w ścisłej kolejności wynikającej z zależności kluczy obcych. Każdy plik tworzył jedną tabelę wraz z constraintami i indeksami.

Kolejność wykonania uwzględniała hierarchię zależności:

- Najpierw tabele bez kluczy obcych: `dyspozytorzy`, `karetki`, `pracownicy`, `zespoly`, `zmiany`, `katalog_swiadczen`, `pacjenci`
- Następnie tabele z FK do już istniejących tabel: `sklad_zespolu`, `dyspozytorzy_zmiany`, `wezwania`
- Na końcu tabele zależne od wielu innych: `wyjazdy`, `wyjazd_pacjent`, `udzielona_pomoc`

Po wykonaniu DDL dane testowe wstawiono skryptami DML w folderze `/sql/dml/`, zachowując tę samą kolejność. Tabele wypełniono danymi: 5 dyspozytorów, 8 karetek trzech typów, 15 pracowników, 6 zespołów, 20 wezwań z różnymi priorytetami i statusami, 15 wyjazdów, 10 pacjentów, 20 świadczeń medycznych oraz dane udzielonej pomocy powiązane z wyjazdami.

8.3 Implementacja zapytań, widoków i wyzwalaczy

Osiem zapytań SQL (Q1-Q8) zaimplementowano jako osobne pliki w katalogu `/sql/queries/`. Każde zapytanie jest parametryzowalne - wartości filtrów (PESEL pacjenta, numer rejestracyjny karetki, miesiąc) można modyfikować bez zmiany struktury zapytania.

Trzy widoki (`v_szczegoly_wyjazdu`, `v_historia_pacjenta`, `v3_obciazenie_dyspozytorni`) utworzono z użyciem składni `CREATE OR REPLACE VIEW`, co umożliwia bezpieczne ponowne wykonanie skryptu bez konieczności uprzedniego usuwania widoku. Widoki zostały zweryfikowane przez wykonanie zapytań SELECT filtrujących wyniki według różnych kryteriów.

Trzy wyzwalacze zaimplementowano w dwóch krokach - najpierw funkcja triggera (`CREATE OR REPLACE FUNCTION`), następnie sam wyzwalacz (`CREATE OR REPLACE TRIGGER`). Poprawność działania każdego triggera zweryfikowano przez wykonanie operacji wyzwalającej i sprawdzenie efektów w tabelach docelowych.

8.4 Implementacja systemu ról i uprawnień

Role zaimplementowano w pliku `/sql/roles/01_role_uprawnienia.sql`. Skrypt rozpoczyna się od usunięcia istniejących ról (`DROP ROLE IF EXISTS`), co zapewnia idempotentność - możliwość bezpiecznego wielokrotnego wykonania bez błędów.

Uprawnienia przyznano zgodnie z zasadą najmniejszych uprawnień. Dodatkowo ograniczono dostęp do schematu `public`:

```
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO dyspozytor, ratownik, admin_pogotowie;
```

Poprawność konfiguracji uprawnień zweryfikowano poprzez przełączanie kontekstu sesji (`SET ROLE`) i wykonywanie operacji dozwolonych i niedozwolonych dla każdej roli.

9. Uwagi końcowe i wnioski

Projekt pozwolił na praktyczne zastosowanie teorii relacyjnych baz danych w realistycznym scenariuszu domenowym. Kluczowe wnioski wyciągnięte w trakcie realizacji:

Normalizacja i projektowanie schematu - Prawidłowe zaprojektowanie struktury tabel w 3NF wymaga starannej analizy zależności funkcyjnych przed napisaniem pierwszej linii SQL. Późniejsze zmiany struktury (np. modyfikacja constraintów CHECK) pociągają za sobą konieczność aktualizacji danych we wszystkich tabelach zależnych, co jest kosztowne i ryzykowne.

Kolejność operacji - Zależności kluczy obcych narzucają ścisłą kolejność zarówno przy tworzeniu tabel (DDL), jak i wstawianiu danych (DML). Naruszenie tej kolejności skutkuje błędami integralności referencyjnej, których diagnoza wymaga dobrego rozumienia modelu danych.

Indeksowanie - Indeksy na polach kluczy obcych i polach wyszukiwania są krytyczne dla wydajności przy dużych wolumenach danych. W projekcie zdefiniowano indeksy na FK, co znacząco przyspieszy zapytania JOIN i filtrowanie.

Triggery jako narzędzie automatyzacji - Wyzwalacze skutecznie eliminują konieczność ręcznej aktualizacji statusów przez użytkownika, jednak należy je projektować ostrożnie - nadmiarowe triggery mogą prowadzić do trudnych do diagnozowania efektów ubocznych i obniżenia wydajności.

Kontrola dostępu - System ról PostgreSQL umożliwia precyzyjne modelowanie uprawnień zgodne ze strukturą organizacyjną instytucji. Zasada najmniejszych uprawnień powinna być stosowana od początku projektu - jej późniejsze wprowadzenie w dojrzałym systemie jest znacznie trudniejsze.

10. Bibliografia / Źródła

1. PostgreSQL 16 Documentation - https://www.postgresql.org/docs/16/
2. Coronel C., Morris S., *Database Systems: Design, Implementation & Management*, Cengage Learning, 2018
3. Date C.J., *An Introduction to Database Systems*, 8th ed., Addison-Wesley, 2004
4. DBeaver Documentation - https://dbeaver.com/docs/
5. dbdiagram.io Documentation - https://dbdiagram.io/docs
6. Momjian B., *PostgreSQL: Introduction and Concepts*, Addison-Wesley, 2001 - dostępne online: https://momjian.us/main/writings/pgsql/aw_pgsql_book/
7. GeeksForGeeks, *How to Design a Database for Healthcare Management System* - https://www.geeksforgeeks.org/dbms/how-to-design-a-database-for-healthcare-management-system/
8. ERflow, *Database Schema Design Best Practices* - https://erflow.io/en/blog/database-schema-design-best-practices